

Command Line Argument ?

Whenever we pass any argument to the **main method then It is called Command Line Argument.**

Example :

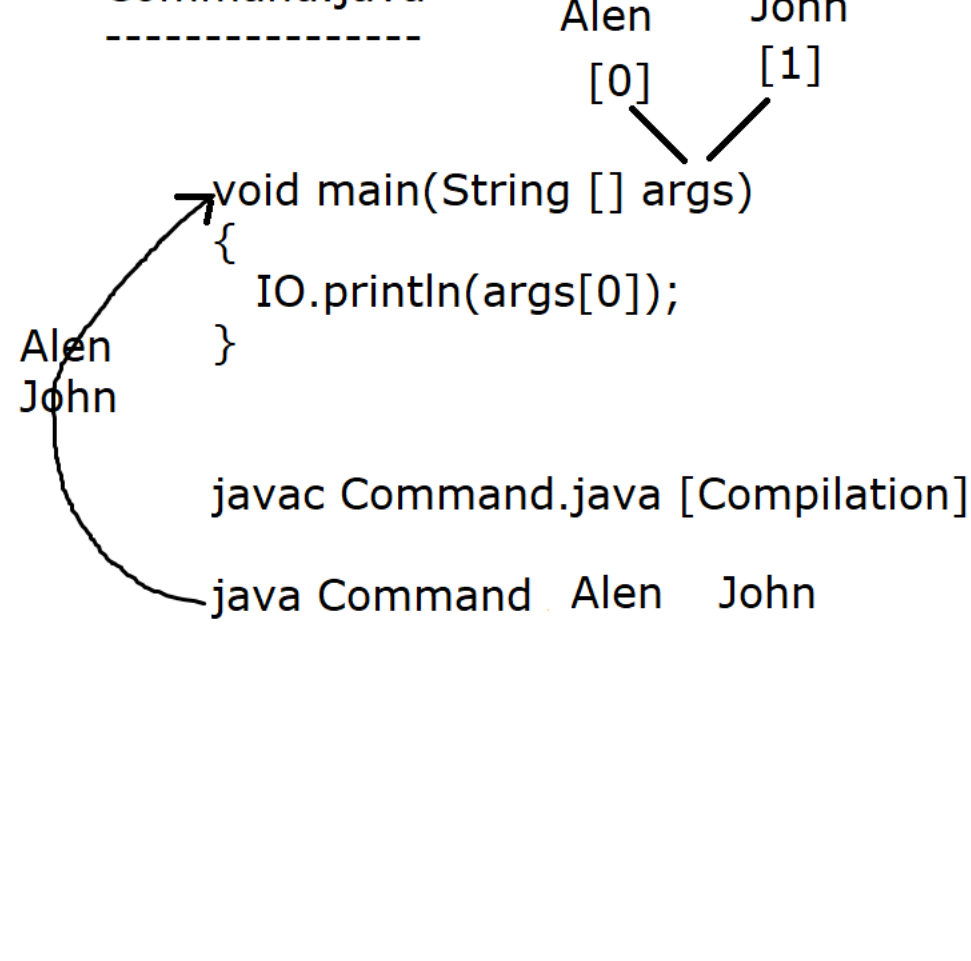
```
void main(String [] args)
{
}
```

* By using command line argument, We can pass some value at runtime i.e. at the time of execution.

* The advantage of command line argument is, single time compilation and number of times **execution** with different values.

* In JDK 25V, Java has provided a non static main method for command line argument which shown below :

```
Example :
void main(String [] args)
{
}
```



//Programs on command line argument :

```
Command.java
-----
void main(String []args)
{
    IO.println(args[0]);
}
```

javac Command.java

java Command Alen John

Output is : Alen

How to configure edit plus for accepting runtime values for Command line argument :

click on add tool

- In the menu text -> Command
- Command -> C:\Program Files\Java\jdk-25\bin\java.exe
- Argument -> File Name without ext Prompt for Argument [%FileNameNoExt] \$(Prompt)]
- File Dir -> Initial Directory

```
Command1.java
-----
void main(String []args)
{
    IO.println(args[1]);
}
```

In the edit plus pass : 12 90

12 90 -> Output is 90

```
Command2.java
-----
void main(String []fruits)
{
    Arrays.sort(fruits);
    for(String fruit : fruits)
    {
        IO.println(fruit);
    }
}
```

In the edit plus pass : Mango Orange Apple Grapes Kiwi

```
Command3.java
-----
Perform addition of two numbers using commad line argument :

void main(String []num)
{
    IO.println(num[0] + num [1]);
}
```

In the edit plus pass : 12 90
1290 is output

```
void main(String []num)
{
    int x = Integer.parseInt(num[0]);
    int y = Integer.parseInt(num[1]);
    int sum = x + y;
    IO.println("Sum is :"+sum);
}
```

In the edit plus pass : 12 90
102 is output

What is Method Overloading in java ?

Allows us to write two or more methods in the same class or in **super and sub classes** (using Inheritance) such that method names are **same**, but they differ in their parameter list which are as follows:

- Number of Parameter
- Type of Parameter
- Order of Parameter (Sequence of data type)

Example:

```
a) Number of Parameter
public void display(int a) { }
public void display(int a, int b) { }
```

```
b) Type of Parameter
public void show(int a) { }
public void show(double a) { }
```

```
c) Order of Parameter (Sequence of data type)
public void test(int a, double b) { }
public void test(double a, int b) { }
```

//Basic Program

Program based on Number of Parameters :

```
MOL1.java
-----
void main()
{
    accept(10,20,30,40);
}
```

```
void accept(int x)
{
    IO.println("One Parameter");
}
```

```
void accept(int x, int y)
{
    IO.println("Two Parameters");
}
```

```
void accept(int x, int y, int z)
{
    IO.println("Three Parameters");
}
```

//Program on MOL on type of parameters :

```
MOL2.java
-----
void main()
{
    show(5);
}
```

```
void show(int a)
{
    IO.println("INT");
}
```

```
void show(double a)
{
    IO.println("DOUBLE");
}
```

//Program on MOL on Order of parameters :

```
MOL3.java
-----
void main()
{
    accept(2,9.0); //int - double
    accept(2.0,9); //double - int
}
```

```
void accept(int x, double y)
{
    IO.println("int - double");
}
```

```
void accept(double x, int y)
{
    IO.println("double - int");
}
```

Rules of Overloading a Method:

- Methods must have the same name.
- Must differ in number, type, or order of parameters.
- Can change the return type.
- Access modifiers can change.
- Static methods can be overloaded.
- Type promotion applies automatically.
- Cannot overload by **only changing return type**.

Note : 1) Method Overloading comes under **static polymorphism** concept.

2) In Java API, **System.out.println()** & **IO.println()** are the best example for Method Overloading